

BUILD SPECIFICATION · WEB PLATFORM

Forge

Learn IT skills, prove them fairly, get hired

The complete brief for Claude Code to build Forge from an empty folder to a launched product, then v1 and v2: vision, architecture, security, data model, problem format, and 31 build phases, each with steps, acceptance tests and a ready-to-paste prompt.

Working name	Forge (placeholder; choose the final name before buying domains)
Product	Web platform for IT skills: Practice, Competitive, Career, Community, Business
Stack	TypeScript monorepo: Next.js web app, NestJS API, PostgreSQL, Redis, isolated code-runner hosts
Scope in this spec	Launch (MVP), v1, v2. v3 (system design, tournaments at scale, enterprise) is out of scope
Spec version	1.0 · 30 September 2026

How to use this document. Put it in an empty project folder as docs/SPEC.pdf and copy Appendix A into CLAUDE.md. Then paste phase prompts into Claude Code one at a time, in order. A phase is finished only when its acceptance tests pass. Commit after every phase.

Be realistic about size. This is a multi-year product. Claude Code can write most of the code, but you must review it, run the servers, pay for infrastructure, check problem quality, handle legal and support, and test with real users. Launch alone is several months of focused work.

Contents

Part	Section
1	Product summary
2	Architecture and repository
3	Security requirements
4	Data model
5	Problem packages (content format)
6	Code runner and live labs
7	Integrity system (verified mode)
8	Ratings, XP and progress
9	Design system and UX rules
10	Plans, billing and entitlements
11	Testing, CI and quality gates
L	Launch phases L0-L13
V1	Version 1 phases V1.1-V1.9
V2	Version 2 phases V2.1-V2.9
12	What you must do yourself
13	Decisions made for you
A	CLAUDE.md

1. Product summary

Forge is a web platform where people learn every kind of IT skill, prove it under fair conditions, and get hired for it. It covers algorithms, debugging, code review, SQL, security, Linux and DevOps, backend, frontend and AI-era skills.

The five pillars

Pillar	Purpose	Key features
Practice	Learn freely	Language courses, pattern lessons, visualizers, hint ladder, editorials, review queue, skill map
Competitive	Prove it; results count	Verified challenges, daily challenge, contests, duels, ratings, seasons
Career	Turn skills into jobs	Public profile, certificates, interview tracks, CV export
Community	Keep people engaged, grow content	Discussions, solutions, study groups, creator program
Business	Main revenue	Hiring assessments, team training, classrooms

Principles every phase must respect

- **Practice is relaxed; Competitive is verified.** AI help is fine in Practice. Only Competitive results change ratings and appear as verified.
- **Real-work skills matter as much as algorithms.** Debugging, reviewing and fixing are first-class formats.
- **No problem is AI-proof,** so verification relies on unique instances, session replay, follow-up questions and an integrity score, never on single-signal bans.
- **Untrusted code never runs next to user data.** The runner is a separate system on separate hosts.
- **Fair to users.** No webcam, no dark patterns, clear consent, easy data export and deletion.

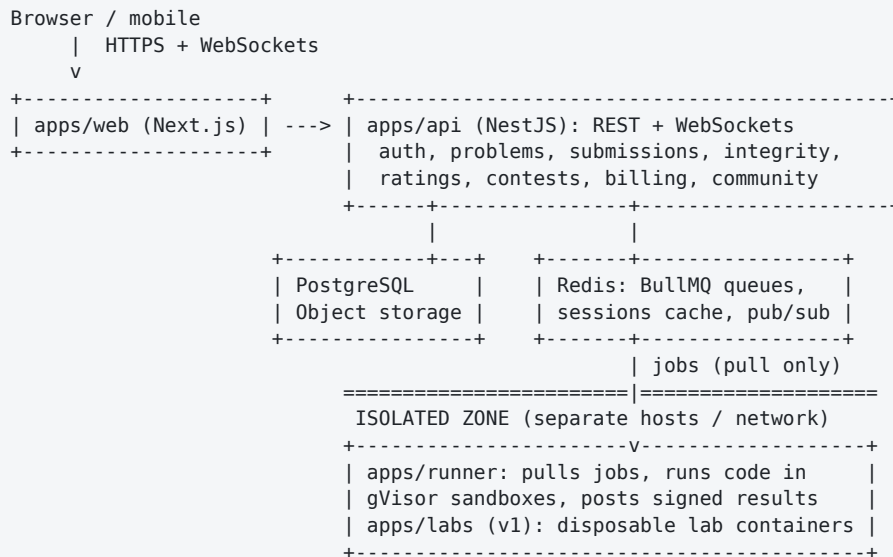
Release scope

Release	Main content
Launch (MVP)	Accounts, Practice (Python, JavaScript/TypeScript, SQL), tracks Algorithms, Debugging, SQL, Security (static), about 150 problems, courses and pattern lessons, hint ladder, editorials, verified challenges with integrity system, daily challenge, streaks, Pro subscription
v1	Ratings, weekly contests, duels, seasons, Java, C++, Bash, code review, backend and AI-era tracks, live Linux and security labs, public profiles, certificates, community discussions, hiring assessments
v2	Mobile companion app, creator program, study groups and clubs, frontend track, Go, C#, Rust, Kotlin, C, teams and classrooms, AI features (Pro Plus), team battles, CTF events, optimisation arenas, spectating

2. Architecture and repository

One TypeScript monorepo. The web app and API run on the main servers; code runners and labs run on separate, locked-down hosts that only talk to the API through a job queue and a signed callback.

System overview



Technology choices

Part	Choice	Notes
Language	TypeScript (strict) everywhere except sandbox images	One language keeps the monorepo simple
Monorepo	pnpm workspaces + Turborepo	Shared types and schemas across apps
Web app	Next.js (App Router), React, Tailwind CSS, shadcn/ui, Monaco editor, xterm.js	Server components for public pages (SEO), client components for the workspace
API	NestJS on Node.js LTS, REST (OpenAPI generated) + WebSockets (Socket.IO with Redis adapter)	Validation with zod schemas from packages/shared
Database	PostgreSQL + Prisma ORM, migrations committed	Row-level checks in code; no raw SQL from user input
Queues and cache	Redis + BullMQ	Submission jobs, emails, rating updates, scheduled tasks
Files	S3-compatible object storage (MinIO in development)	Replays, challenge files, avatars; private buckets, signed URLs
Sandbox	Docker with the gVisor (runc) runtime, one image per language	Network off, read-only root, seccomp, cgroup limits
Payments	Stripe Billing + Checkout + Customer Portal + Stripe Tax	Webhooks verified by signature
Email	One transactional provider behind an interface (SMTP in development)	Verification, resets, receipts, digests

Part	Choice	Notes
Search	PostgreSQL full-text search	Upgrade later only if needed
Mobile (v2)	Expo (React Native) sharing packages/shared	Android and iOS from one codebase
Hosting	EU region. Docker Compose on VMs at launch; runners on separate VMs	Move to Kubernetes only when scale requires

Repository layout

Folder structure

```

forge/
  apps/
    web/      Next.js app (public pages, dashboard, workspace, admin)
    api/      NestJS API (modules: auth, users, problems, submissions, practice,
              integrity, competitive, ratings, billing, community, orgs, admin)
    runner/   Job consumer that executes code in sandboxes (isolated hosts only)
    labs/     (v1) Lab orchestrator for live terminals and vulnerable apps
    mobile/   (v2) Expo app
  packages/
    shared/   zod schemas, API types, constants (plans, limits, tracks, languages)
    db/       Prisma schema, migrations, seed scripts
    problem-kit/ Problem package format, generator runtime, validator CLI
    ui/       Shared React components and design tokens
    config/   ESLint, TypeScript, Tailwind presets
  sandboxes/ Dockerfiles per language (python, node, postgres-sql, ...)
  content/   Problem packages, courses, lessons (versioned in git)
  infra/     docker-compose files, deployment scripts, backup scripts
  docs/      SPEC.pdf, decisions/ (ADRs), SECURITY_REVIEW.md, runbooks

```

API conventions

- REST under `/api/v1`, JSON, OpenAPI document generated from the code and published at `/api/docs` in development only.
- Every request body, query and response is validated with zod schemas shared from `packages/shared`.
- Errors use one shape: `{ error: { code, message, details? } }` with stable codes (for example `PLAN_REQUIRED`, `RATE_LIMITED`).
- Pagination is cursor-based. Lists never return more than 100 items.
- Idempotency keys on every create action that could be retried (submissions, purchases, invites).
- All times stored in UTC; users choose a display time zone.

3. Security requirements

These apply to every phase. Phase L12 and each release's hardening phase verify all of them with evidence.

3.1 Accounts and sessions

- Passwords hashed with argon2id; minimum 10 characters; checked against a list of known breached passwords (k-anonymity range check or a bundled list).
- Sessions are server-side, referenced by an HttpOnly, Secure, SameSite=Lax cookie; rotated on login and privilege change; revocable from Settings.
- OAuth sign-in with GitHub and Google using PKCE and the state parameter.
- Email verification required before submitting to Competitive or posting in Community.
- Optional TOTP two-factor authentication for everyone; required for admins, moderators and company owners.
- Login, signup, password reset and verification are rate-limited per IP and per account, with lockout backoff.

3.2 Web and API

- CSRF protection on all state-changing requests (SameSite cookies plus a CSRF token header).
- Strict Content-Security-Policy with nonces; no inline scripts; frame-ancestors none except the lab and preview iframes, which use a separate sandbox domain.
- User-generated content (markdown in discussions, solutions, problem statements) is rendered with a strict sanitizer; no raw HTML.
- Authorization checked in the API for every resource (ownership, organization membership, role, plan). Never rely on the UI hiding things.
- Rate limits on every endpoint class (reads, writes, submissions, searches), stricter for anonymous users.
- Security headers: HSTS, X-Content-Type-Options, Referrer-Policy, Permissions-Policy.

3.3 Secrets and infrastructure

- No secrets in the repository. Local development uses .env files that are git-ignored; production uses the host's secret store.
- The runner hosts have no database credentials. They pull jobs from a dedicated Redis with a restricted user and return results by signed HTTPS callback (HMAC with a rotating key).
- Database, Redis and object storage are not reachable from the internet.
- Admin panel requires 2FA, is audit-logged, and uses separate admin roles (content editor, moderator, support, superadmin).
- Automated dependency scanning and secret scanning in CI; high or critical findings block merges.
- Daily encrypted database backups, kept 30 days, with a tested restore procedure.

3.4 Content and flags

- Hidden tests, reference solutions and competitive templates never leave the server.
- Security flags are per user: `FORGE{HMAC(secret, userId + challengeId)[0:24]}`. A flag submitted by the wrong user is recorded as a sharing signal.
- Downloadable challenge files are served through short-lived signed URLs.

3.5 Privacy (GDPR)

- Data stored in the EU. A privacy policy, terms of service and cookie notice (only essential cookies at launch).
- Users can export all their data (JSON + files) and delete their account; deletion completes within 30 days, backups roll off within 30 more.
- Session recording for verified challenges requires explicit consent each time; replays are deleted after 12 months unless the user keeps them public.
- Minimum age 16. Under-18 users are hidden from company search and can't receive direct messages from strangers.

4. Data model

Core tables at launch (Prisma models). Later phases add tables noted with their phase. All tables have id (UUID), createdAt and updatedAt.

Model	Key fields	Phase
User	email (unique), emailVerifiedAt, passwordHash?, displayName, handle (unique), avatarKey, country, timeZone, role, birthYear, deletedAt	L2
Session / OAuthAccount / TotpSecret	user, expiresAt, userAgent, ip hash / provider, providerUserId / encrypted secret	L2
Track	slug, name, order	L3
Problem	slug, track, title, difficulty, rating, tags[], formats[], languages[], mode (practice competitive both), status, version	L3
ProblemVersion	problem, version, packageHash, statement, limits, publishedAt	L3
Submission	user, problem, version, language, code, kind (run submit), status, verdict, runtimeMs, memoryKb, testsPassed, testsTotal, attemptId?	L4
Course / Lesson / LessonProgress	track or language, order, content key / user, lesson, completedAt	L6
HintUse / Bookmark / Note	user, problem, level / user, problem / user, problem, text	L6
Mastery	user, concept tag, level (0-5), lastPracticed, nextReviewAt	L6
FlagChallenge / FlagSubmission	problem, fileKeys / user, problem, value hash, correct, sharedFrom?	L7
Attempt (verified)	user, problem, seed, startedAt, endsAt, submittedAt, score, integrityScore, status (verified unverified review appealed)	L8
AttemptEvent	attempt, t (ms), type, payload (compressed)	L8
FollowUp	attempt, kind, prompt, expected, answer, correct, answeredMs	L8
Review / Appeal	attempt, reviewer, decision, notes / attempt, reason, outcome	L8
DailyChallenge / Streak / XpEvent	date, track, problem / user, current, longest, freezes / user, amount, reason	L9
Subscription / Entitlement	user or org, stripeCustomerId, stripeSubId, plan, status, periodEnd / subject, feature, until	L10

Model	Key fields	Phase
AuditLog	actor, action, target, ip hash, at	L2
Rating / RatingChange	user, track, rating, rd, volatility / event, before, after	V1.1
Contest / ContestEntry / Duel	schedule, problems / user, score, rank / players, problem, result	V1.2-V1.3
Season / LeagueGroup	start, end / season, members	V1.4
Lab / LabSession	image, limits / user, host, token, expiresAt	V1.6
Certificate	user, track, level, issuedAt, expiresAt, verifyCode	V1.7
Post / Comment / Vote / Report	problem or topic, author, body, status / target, reason	V1.8
Organization / Membership / Assessment / Invite	name, plan / user, role / problems, settings / email, token, status	V1.9
Group / Club / Follow / Message	social features	V2.3
CreatorProfile / Payout	user, stripe account / amount, period	V2.2
Classroom / Assignment / Grade	org, teacher / problems, due / student, score	V2.6
AIUsage	user, feature, tokens, cost, at	V2.7

5. Problem packages (content format)

Every problem is a folder in `content/problems/`, validated by the `problem-kit` CLI and imported into the database. Content lives in git so changes are reviewed like code.

Package layout

```
content/problems/two-sum-orders/
  problem.yaml      metadata (below)
  statement.md      text with {{placeholders}} filled from the generated instance
  generator.ts       (seed) => instance: data, names, planted bug, expected results
  tests.ts          (instance) => visible tests + hidden tests
  reference/        correct solutions per language (never shown to users)
    solution.py     solution.ts
  starter/          starter code per language (fix-code formats include the bug)
  hints.md          exactly 4 levels: nudge, approach, pseudocode, solution
  editorial.md       idea, why it works, complexity, common mistakes
  followups.ts       (instance, userCode) => follow-up questions (verified mode)
```

problem.yaml

```
id: two-sum-orders
title: Matching Orders
track: algorithms
format: write-code          # write-code | fix-code | review | predict | flag | terminal | build
difficulty: easy            # easy | medium | hard | expert
tags: [hashing, arrays]
languages: [python, typescript, javascript]
mode: both                  # practice | competitive | both
limits: { timeMs: 2000, memoryMb: 256, outputKb: 64 }
authors: [forge-team]
version: 3
```

Validator rules (problem-kit validate)

- Runs every reference solution against tests for at least 50 random seeds; all must pass within limits.
- Runs starter code and known-wrong solutions (in wrong/) and confirms they fail, so tests actually catch mistakes.
- Checks the generator is deterministic (same seed, same instance) and that instances really differ between seeds.
- Checks statement placeholders all resolve, hints have 4 levels, editorial exists, and tags are from the allowed list.
- Fails the CI build if any problem is invalid.

Other content types

- **Lessons:** MDX files in content/courses/<course>/ with frontmatter (title, order, concepts) and embedded exercise blocks that reference a small problem package.
- **Visualizers:** React components in apps/web driven by a step list produced by instrumented reference code.
- **Flag challenges:** the package holds the downloadable files and a generator that embeds the per-user flag where needed.
- **Review challenges:** a diff plus a list of planted issues (line range + issue type) to grade against.

Content is the product. Claude Code can draft problems in this format, but every problem must be read and solved by a human before publishing. Budget real time for this: 150 good problems is weeks of review work.

6. Code runner and live labs

The runner is the highest-risk component. It executes code written by strangers, so it is designed as if every submission is an attack.

6.1 Runner design

- Runs on dedicated VMs with no database access, no cloud credentials, and outbound traffic blocked except to the job Redis and the API callback URL.
- Pulls jobs from BullMQ; each job contains language, code, tests and limits, signed by the API.
- Each run starts a fresh container from a pinned per-language image using the gVisor runtime:
 - network none, read-only root filesystem, a small tmpfs work directory, non-root user, all capabilities dropped, seccomp default profile, no-new-privileges.
- Limits: CPU time and wall time, memory, process count (pids limit), output size, file size. Exceeding a limit returns a clear verdict (Time Limit, Memory Limit, Output Limit).
- Containers are destroyed after every run; nothing is reused between users.
- Results are posted back with an HMAC signature and a job ID; the API rejects unsigned or replayed results.
- SQL problems run against a disposable PostgreSQL inside the sandbox, loaded with the instance's data, with a statement timeout.

6.2 Verdicts

Verdict	Meaning
Accepted	All tests pass within limits
Wrong Answer	At least one test output differs (visible tests show the diff; hidden tests show only which category failed)
Time Limit / Memory Limit / Output Limit	A limit was exceeded
Runtime Error	The program crashed; the error message is shown for visible tests
Compile Error	The code didn't compile (compiled languages)
Internal Error	Runner fault; the submission is retried once and never counts against the user

6.3 Live labs (v1)

- A separate orchestrator on separate lab hosts starts one container per user session from a lab image, on its own isolated network with no internet access.
- The user reaches it through a WebSocket terminal gateway (xterm.js) or, for web-app labs, a reverse proxy on a separate lab domain, both authenticated with a short-lived signed token.
- Sessions end after 60 minutes (extendable once) or 15 minutes idle, and the container and its network are destroyed.
- Checks for terminal tasks run inside the lab and report the target state to the orchestrator.
- Per-user and global concurrency caps protect costs.

6.4 Security testing of the runner

Phase L4 must include an attack test suite that tries to: open network connections, read host files, fork-bomb, fill the disk, allocate huge memory, run forever, escalate privileges, and escape via known container techniques. Every test must be contained and produce the right verdict, with the host unaffected.

7. Integrity system (verified mode)

Verified challenges are how Forge earns employer trust. The system makes cheating slow, risky and visible, and never bans on a single signal.

7.1 Attempt flow

- 1 Consent screen listing what's recorded; no webcam.
- 2 Server creates an Attempt with a random seed; the generator builds a unique instance; hidden tests and expected answers are computed server-side.
- 3 Full-screen workspace with a visible timer (the problem's time budget, default 45 minutes).
- 4 The client records events and sends them in batches every 5 seconds; the server stores them compressed.
- 5 On submit: hidden tests, quality checks (linters, security scanners), then 2–3 follow-up questions, 90 seconds each.
- 6 The integrity score is computed; the attempt becomes verified, unverified, or sent to review.

7.2 Recorded events

Event	Payload
edit	Changes as small diffs with timestamps (enough to replay typing)
paste	Length, whether the content came from inside the editor, first 200 characters hashed
run / test / submit	Code hash and results
focus / blur / visibility	Time the workspace lost and regained focus
fullscreen exit	Timestamp
follow-up answer	Question ID, answer, time taken

7.3 Follow-up question types

- **Predict:** "What does your function return for this input?" The server runs the user's own code to get the true answer.
- **Edge case:** "Does your code handle an empty list correctly? Yes or no." The server tests it.
- **Change:** "If the rule changed to X, which line would you edit?" Graded against lines that depend on that rule.
- **Explain choice:** multiple choice about a structure the user actually used (map, sort, recursion).

7.4 Integrity score

A 0–100 score from weighted signals. Weights live in one config file and are tuned with real data. Starting weights:

Signal	Effect
Follow-up accuracy	Strongest positive signal; all wrong is the strongest negative
Large paste from outside the editor	Negative, scaled by size relative to the final solution
Time out of focus	Negative beyond 60 seconds total
Solve time far below the problem's typical time	Negative (compared with the distribution of verified solvers)
Natural editing history (incremental edits, runs, fixes)	Positive
Account trust (age, verified email, history)	Small positive

- Score 70 or more: **verified**. 40–69: **unverified** (result shown, not counted). Below 40: **review** by a human moderator.
- Users can appeal any non-verified result; a reviewer watches the replay and decides.
- Templates are retired when leaks are detected; competitive templates never appear in Practice.

8. Ratings, XP and progress

System	Rules
Track rating (v1)	Glicko-2 per user per track, starting at 1500. Updated after contests, duels and verified challenges. Hidden until 5 rated events (placement).
Problem rating	Starts from difficulty (Easy 1000, Medium 1400, Hard 1800, Expert 2200), recalculated weekly from verified solve rates.
Mastery (launch)	Per concept tag, levels 0–5, from Practice results; drives the skill map and recommendations.
Review queue (launch)	Struggled problems return after 2, 7 and 21 days (simplified spaced repetition).
XP (launch)	Solve: 10/20/40/80 by difficulty, minus 25% per hint level used. Lessons: 5. Daily challenge: +10. No XP for Competitive losses.
Streaks (launch)	A day counts with one solved problem or lesson, in the user's time zone. Free: 2 freezes/month; Pro: 5.
Leagues (v1)	Weekly XP groups of 30 with promotion/relegation.

9. Design system and UX rules

Area	Rule
Look	Modern, calm developer tool. Dark theme by default with a full light theme. Neutral greys, one brand accent (suggested electric blue #3B82F6), status colours for verdicts.
Typography	Inter (UI) and JetBrains Mono (code), both open-source and self-hosted
Components	shadcn/ui on Radix primitives, customised tokens in packages/ui; no one-off styles in pages
Workspace layout	Resizable panes: statement (left), editor (right top), console and tests (right bottom). Remembers pane sizes. Collapses to tabs on narrow screens
Performance	Public pages Largest Contentful Paint under 2.5 s on 4G; editor ready under 2 s after navigation
Accessibility	WCAG 2.2 AA: keyboard-only use, visible focus, screen-reader labels, 4.5:1 contrast, reduced-motion support, captions on videos
Copy	Plain, friendly English; error messages say what happened and what to do. Romanian translation in v1
Empty and loading states	Every list has an empty state with one action; skeleton loaders, never blank screens
Responsive	Everything except the full workspace works on phones; the workspace shows a clear note suggesting a larger screen

10. Plans, billing and entitlements

Plan	Price (suggested)	Includes	Phase
Free	€0	Daily challenge, first lessons of each course, 3 hint levels per day, all verified challenges (3 per week), contests and duels (v1)	L10
Pro	€12/month or €99/year	Everything in Practice, unlimited hints, editorials, review queue, study plans, unlimited verified challenges, 5 streak freezes	L10
Student	50% off Pro	University email verification	V1.9
Teams	€25/seat/month	Team dashboards, assigned paths	V2.6
Hiring	From €299/month	Assessments, candidate results and replays	V1.9
Education	Per student per year	Classrooms, assignments, exam mode	V2.6
Pro Plus	€24/month or €199/year	Pro + AI tutor and reviews	V2.7

- Plan limits live in packages/shared/plans.ts and are enforced only in the API through one EntitlementsService. The web app reads entitlements to show locks, never to decide.

- Stripe webhooks (checkout completed, subscription updated/deleted, invoice paid/failed) update Subscription and Entitlement records. Webhooks are verified, idempotent and processed through a queue.
- Regional prices through Stripe prices per currency/country (Romania and similar markets 40–60% lower).
- Stripe Tax handles EU VAT; invoices available in Settings.
- Failed payments get a 7-day grace period before downgrade.
- The mobile app (v2) does not sell subscriptions in-app; users subscribe on the website. Check store rules before adding any purchase link inside the app.

11. Testing, CI and quality gates

Layer	Tools	Must cover
Unit	Vitest	Services, entitlements, integrity scoring, rating math, generators
API integration	Vitest + Supertest against a real PostgreSQL and Redis in Docker	Every endpoint: auth, permissions, validation errors, rate limits
End-to-end	Playwright	Sign up → solve → submit; verified attempt; checkout (Stripe test mode); admin review
Runner	Dedicated test suite on a runner VM or local gVisor	Verdicts per language and the attack suite in 6.4
Content	problem-kit validate	Every problem package
Security	Dependency audit, secret scanning, OWASP ZAP baseline scan in CI	No high/critical findings
Accessibility	axe checks in Playwright	No serious violations on main pages

- CI (GitHub Actions) runs lint, type-check, unit, integration, content validation and e2e on every pull request.
- Main branch is protected; merges only with green CI.
- Preview deployments are optional; a staging environment mirrors production and uses Stripe test mode.

PART L

Launch (MVP)

Everything needed to open Forge to the public: accounts, Practice with three languages and four tracks, a secure runner, verified challenges with the integrity system, daily challenge and streaks, and the Pro subscription. Target: about 150 high-quality problems and 3 short courses.

Phase L0 — Accounts and tools (you do this)

Goal: have everything Claude Code needs before it starts. (Spec: 12)

Build steps

- 1 Choose a working name and check the domain is available; buy it.
- 2 Create a GitHub account and an empty private repository; install Git, Node.js LTS, pnpm, Docker Desktop, and Claude Code.
- 3 Create a Stripe account (test mode is enough for now) and a transactional email provider account.
- 4 Rent or plan hosting in an EU region: one VM for the app, one separate VM for the runner (you can start both locally).
- 5 Put this PDF in docs/SPEC.pdf and Appendix A in CLAUDE.md, then commit.

Done when (acceptance criteria)

- Repository exists with docs/SPEC.pdf and CLAUDE.md.
- Docker runs locally.

Paste into Claude Code

Read CLAUDE.md and docs/SPEC.pdf fully. Don't write code yet.
Summarise the product in 15 bullets, list every ambiguity or conflict you find, and propose any changes to the phase order with reasons.
Re-read CLAUDE.md first. Finish by running: `pnpm lint && pnpm test && pnpm test:e2e`, and list anything unfinished or needing my action.

Phase L1 — Monorepo skeleton

Goal: a clean, fully tooled monorepo where every app starts and CI is green. (Spec: 2, 9, 11)

Build steps

- 1 Create the pnpm + Turborepo workspace with the folder layout from section 2.
- 2 Scaffold apps/web (Next.js, Tailwind, shadcn/ui, dark/light theme, Inter + JetBrains Mono self-hosted) and apps/api (NestJS with health endpoint, zod validation pipe, error format).
- 3 Create packages/shared, packages/db (Prisma, PostgreSQL), packages/ui, packages/config.
- 4 Docker Compose for development: PostgreSQL, Redis, MinIO, Mailpit (local email).
- 5 ESLint, Prettier, strict TypeScript, Vitest, Playwright, and a GitHub Actions workflow running lint, type-check and tests.
- 6 Security headers and CSP skeleton in web and API. `.env.example` files documented; real `.env` files git-ignored.

Done when (acceptance criteria)

- `pnpm dev` starts web, API and services; the web app calls the API health endpoint and shows the result.
- CI passes on a pull request.
- A Playwright smoke test loads the home page in both themes.

Paste into Claude Code

Do phase L1 from docs/SPEC.pdf. Use current stable versions of every tool (look them up; never invent versions or use pre-releases).
Re-read CLAUDE.md first. Finish by running: `pnpm lint && pnpm test && pnpm test:e2e`, and list anything unfinished or needing my action.

Phase L2 — Authentication and accounts

Goal: secure sign-up, sign-in and account management. (Spec: 3.1, 3.5, 4)

Build steps

- 1 User, Session, OAuthAccount, TotpSecret and AuditLog models with migrations.
- 2 Email/password sign-up with argon2id, breached-password check, email verification, password reset.
- 3 GitHub and Google OAuth with PKCE and state; account linking by verified email only.
- 4 Server-side sessions, device list and revoke in Settings; TOTP 2FA with recovery codes.
- 5 Rate limiting and lockout backoff on all auth endpoints.
- 6 Onboarding: handle, display name, country, time zone, birth year (block under 16), goal and languages.
- 7 Settings: profile, password, 2FA, sessions, data export (JSON download), account deletion (with 30-day process).
- 8 Admin roles and an audit log for sensitive actions.

Done when (acceptance criteria)

- Integration tests: sign-up, verification, login, wrong password lockout, reset, OAuth callback (mocked), 2FA, session revoke, deletion.
- CSRF and rate-limit tests pass.
- Under-16 birth year is blocked.

Paste into Claude Code

Do phase L2 from docs/SPEC.pdf. Write the security tests first, then implement.
Re-read CLAUDE.md first. Finish by running: `pnpm lint && pnpm test && pnpm test:e2e`, and list anything unfinished or needing my action.

Phase L3 — Problem packages and library

Goal: a validated content pipeline and a browsable problem library. (Spec: 5, 4)

Build steps

- 1 Build packages/problem-kit: package schema (zod), generator runtime with seeded random, validator CLI with every rule in section 5.
- 2 Track, Problem and ProblemVersion models; an import command that validates and upserts packages from content/.
- 3 Write 6 sample problems (2 algorithms, 2 debugging, 1 SQL, 1 flag) covering every launch format, each fully valid.
- 4 Problem list page with filters (track, difficulty, tags, status solved/unsolved) and search; problem page showing the statement for a Practice instance.
- 5 Admin pages: list problems, publish/unpublish, see validation reports.

Done when (acceptance criteria)

- `pnpm problems:validate` passes for all samples and fails for a deliberately broken package (test).
- Import is idempotent.
- Hidden tests and reference solutions are never returned by any endpoint (test).

Paste into Claude Code

Do phase L3 from docs/SPEC.pdf. The runner doesn't exist yet: validate reference solutions with a local, clearly marked development executor that is never used in production. Re-read CLAUDE.md first. Finish by running: `pnpm lint && pnpm test && pnpm test:e2e`, and list anything unfinished or needing my action.

Phase L4 — Secure code runner

Goal: run untrusted Python, JavaScript/TypeScript and SQL safely. (Spec: 6.1, 6.2, 6.4)

Build steps

- 1 Sandbox images in `sandboxes/` for Python, Node (JS/TS via a fast transpiler), and PostgreSQL SQL, pinned by digest.
- 2 `apps/runner`: BullMQ consumer, gVisor container per run with all restrictions from 6.1, test harness per language, verdict calculation, signed result callback.
- 3 API side: Submission model, submit/run endpoints, job signing, callback verification, per-user submission rate limits, retry on Internal Error.
- 4 Replace the development executor in `problem-kit` with the real runner (keeping the dev executor only for local content authoring).
- 5 Attack test suite from 6.4.
- 6 Deployment notes for a runner VM: firewall rules, gVisor install, no credentials except Redis and callback key.

Done when (acceptance criteria)

- Every verdict type is produced correctly for each language (tests).
- The attack suite is fully contained and the host stays healthy.
- 100 concurrent submissions complete without errors on a development machine; median run time under 2 seconds for simple problems.

Paste into Claude Code

Do phase L4 from docs/SPEC.pdf. Treat every submission as hostile. Write the attack suite first and show me which attacks are blocked by which restriction. Re-read CLAUDE.md first. Finish by running: `pnpm lint && pnpm test && pnpm test:e2e`, and list anything unfinished or needing my action.

Phase L5 — Workspace

Goal: an excellent editor experience for solving problems. (Spec: 9, 6.2)

Build steps

- 1 Resizable three-pane workspace; Monaco editor with language switching, themes, Vim/Emacs keybindings, font size.
- 2 Run with custom input, run visible tests, submit; results panel with verdicts, diffs, runtime and memory.
- 3 Autosave drafts per problem and language; version history of runs.
- 4 SQL workspace: schema browser, results table, query timing.
- 5 Flag format: download files (signed URLs), submit flag.
- 6 Keyboard shortcuts and accessible labels throughout.

Done when (acceptance criteria)

- Playwright: open problem, write solution, run, fail, fix, submit, Accepted.
- Drafts survive reload.
- axe finds no serious issues on the workspace.

Paste into Claude Code

Do phase L5 from docs/SPEC.pdf. Make the workspace feel as polished as a real IDE.
Re-read CLAUDE.md first. Finish by running: `pnpm lint && pnpm test`
&& `pnpm test:e2e`, and list anything unfinished or needing my action.

Phase L6 — Practice: learning features

Goal: turn the library into a place to learn. (Spec: 1, 5, 8)

Build steps

- 1 Hint ladder (4 levels, daily limits by plan), editorials (locked until solved or given up), "give up" flow.
- 2 Courses and lessons from MDX with embedded runnable exercises; lesson progress.
- 3 Three launch courses: Python basics, JavaScript basics, SQL basics (Claude Code drafts, you review).
- 4 Pattern lessons for 6 core patterns (hashing, two pointers, sliding window, recursion, BFS/DFS, binary search) with pattern drills.
- 5 Three visualizers: array pointers, grid fill (DP), graph search, driven by step lists.
- 6 Mastery per concept, skill map page, review queue (2/7/21 days), bookmarks and private notes.
- 7 Recommendations: "next best problem" from weak concepts, review queue and goal.

Done when (acceptance criteria)

- Tests for hint limits per plan, editorial locking, mastery updates, review scheduling.
- Playwright: complete a lesson, use a hint, solve, see mastery change.

Paste into Claude Code

Do phase L6 from docs/SPEC.pdf. Draft course content in content/courses and mark every drafted lesson with 'status: needs-review' in its frontmatter.
 Re-read CLAUDE.md first. Finish by running: `pnpm lint && pnpm test && pnpm test:e2e`, and list anything unfinished or needing my action.

Phase L7 — Static security challenges

Goal: the security track's first challenges, safely. (Spec: 3.4, 5)

Build steps

- 1 Flag challenge support: per-user HMAC flags, generators that embed flags in files, sharing detection.
- 2 Challenge types: cryptography, forensics (pcap and log files), web vulnerability analysis (read the code, find and explain the bug), and secure fix (fix-code format on vulnerable code).
- 3 Write 10 sample security challenges across these types.
- 4 Clear rules page: only attack Forge's own challenges.

Done when (acceptance criteria)

- A flag from user A submitted by user B is rejected and logged (test).
- No challenge file contains another user's flag (test across seeds).

Paste into Claude Code

Do phase L7 from docs/SPEC.pdf. No real malware; build harmless purpose-made samples only.
 Re-read CLAUDE.md first. Finish by running: `pnpm lint && pnpm test && pnpm test:e2e`, and list anything unfinished or needing my action.

Phase L8 — Verified challenges and integrity

Goal: the core differentiator: verified results. (Spec: 7)

Build steps

- 1 Attempt, AttemptEvent, FollowUp, Review and Appeal models.
- 2 Consent screen, attempt creation with seeds, full-screen timed workspace, event recorder (batched, compressed).
- 3 Follow-up questions (all four types) generated by followups.ts and graded by running the user's code.
- 4 Integrity score calculator with weights in config; verified/unverified/review states.
- 5 Replay viewer: play, pause, speed, timeline of events.
- 6 Moderator review queue and appeal flow in admin.
- 7 Weekly limit of verified challenges on Free.

Done when (acceptance criteria)

- Unit tests for scoring with synthetic event logs (honest solve, pasted solve, tab-away solve).
- Two users starting the same template get different instances (test).
- Playwright: full verified attempt from consent to verified badge.
- Replays are only visible to the owner and moderators (test).

Paste into Claude Code

Do phase L8 from docs/SPEC.pdf. Keep all score weights in one config file and document each signal in docs/decisions/integrity-score.md.
Re-read CLAUDE.md first. Finish by running: `pnpm lint && pnpm test && pnpm test:e2e`, and list anything unfinished or needing my action.

Phase L9 — Daily challenge, streaks, XP and onboarding

Goal: reasons to come back every day. (Spec: 8)

Build steps

- 1 Daily challenge per track (scheduled job picks from a queue of approved problems), same template, unique instance.
- 2 XP events and levels, streaks with freezes in the user's time zone, a small profile summary.
- 3 Onboarding goal and a short placement quiz (10 questions) that sets starting mastery.
- 4 Dashboard home adapted to the user's goal; weekly progress email (opt-in).

Done when (acceptance criteria)

- Streak tests across time zones and midnight boundaries.
- The daily challenge changes at midnight UTC and every user gets a valid instance.

Paste into Claude Code

Do phase L9 from docs/SPEC.pdf. No dark patterns: no guilt notifications, and emails are opt-in with one-click unsubscribe.
Re-read CLAUDE.md first. Finish by running: `pnpm lint && pnpm test && pnpm test:e2e`, and list anything unfinished or needing my action.

Phase L10 — Billing and Pro

Goal: working subscriptions with server-side enforcement. (Spec: 10)

Build steps

- 1 `plans.ts` with every limit; `EntitlementsService` used by every gated endpoint.
- 2 Stripe Checkout for Pro monthly/yearly, Customer Portal, webhooks through a queue, idempotent processing.
- 3 Regional pricing, Stripe Tax, invoices page, 7-day grace on failed payment.
- 4 Pricing page and upgrade prompts at every lock.

Done when (acceptance criteria)

- Integration tests with Stripe test webhooks for every event, including duplicates and out-of-order delivery.
- A Free user can't reach any Pro endpoint by calling the API directly (test for each).

Paste into Claude Code

Do phase L10 from docs/SPEC.pdf using Stripe test mode. List the Stripe dashboard steps I must do by hand.
Re-read CLAUDE.md first. Finish by running: `pnpm lint && pnpm test && pnpm test:e2e`, and list anything unfinished or needing my action.

Phase L11 — Launch content

Goal: about 150 reviewed problems and polished courses. (Spec: 5, 4)

Build steps

- 1 Target mix: Algorithms 60, Debugging 35, SQL 30, Security 25; each track spread across difficulties; 40 of them competitive-enabled templates.
- 2 Claude Code drafts packages in batches of 10 with validation passing; you review, solve and approve each batch.
- 3 Editorials and 4-level hints for every problem.
- 4 A content style guide in docs/content-style.md so problems read consistently.

Done when (acceptance criteria)

- All packages validate in CI.
- Every published problem has status approved by a human reviewer.

Paste into Claude Code

```
Do phase L11 from docs/SPEC.pdf for the next batch of 10 problems in the <TRACK> track, difficulty mix <MIX>. Stop after the batch so I can review it. Re-read CLAUDE.md first. Finish by running: pnpm lint && pnpm test && pnpm test:e2e, and list anything unfinished or needing my action.
```

Phase L12 — Hardening and security review

Goal: confidence before real users arrive. (Spec: 3, 6.4, 11)

Build steps

- 1 Walk through every item of section 3 and record evidence in docs/SECURITY_REVIEW.md.
- 2 Try to break it: IDOR checks on every resource, auth bypass attempts, rate-limit bypasses, XSS in every user field, oversized payloads, replayed webhooks and runner callbacks.
- 3 OWASP ZAP baseline, dependency audit, secret scan; fix all high/critical.
- 4 Load test: 500 concurrent users browsing and 50 submissions per second for 5 minutes on staging.
- 5 Accessibility pass with axe and keyboard-only testing; performance budgets from section 9.
- 6 Legal pages (privacy, terms, cookies, acceptable use) drafted for your lawyer to check.

Done when (acceptance criteria)

- SECURITY_REVIEW.md has every item verified.
- No high/critical findings open.
- Performance and accessibility targets met.

Paste into Claude Code

```
Do phase L12 from docs/SPEC.pdf. Act as a hostile security reviewer. Fix everything you find before reporting, and list anything that needs a human decision. Re-read CLAUDE.md first. Finish by running: pnpm lint && pnpm test && pnpm test:e2e, and list anything unfinished or needing my action.
```

Phase L13 — Deploy and launch

Goal: Forge live in production with monitoring and backups. (Spec: 2, 3.3)

Build steps

- 1 Production Docker Compose (or equivalent) for web, API, workers; separate runner VM; managed or self-hosted PostgreSQL and Redis with private networking.
- 2 TLS, firewall rules, secrets in the host's secret store, automated backups with a restore test.
- 3 Monitoring: uptime checks, error tracking, logs, queue depth and runner health alerts; a public status page.
- 4 Runbooks in docs/runbooks: deploy, rollback, restore backup, rotate keys, runner incident.
- 5 Stripe live mode, production email domain (SPF, DKIM, DMARC), analytics without personal data.
- 6 Soft launch to a small invited group, then public launch.

Done when (acceptance criteria)

- A full deploy and a rollback have been rehearsed.
- A backup restore has been tested.
- Alerts reach you when the runner is stopped (tested).

Paste into Claude Code

Help me with phase L13 from docs/SPEC.pdf. Write the deployment scripts and runbooks, and give me an ordered checklist of the manual steps only I can do.
Re-read CLAUDE.md first. Finish by running: `pnpm lint && pnpm test && pnpm test:e2e`, and list anything unfinished or needing my action.

PART V1

Version 1

Competition, more languages and tracks, live labs, public profiles, certificates, community discussion, and the first company product: hiring assessments. Start only after the launch gate: about 1,000 weekly active users and first paying users.

Phase V1.1 — Ratings

Goal: fair, per-track skill ratings. (Spec: 8)

Build steps

- 1 Glicko-2 implementation in packages/shared with published test vectors.
- 2 Rating and RatingChange models; updates from verified challenges (and later contests and duels) via a queue.
- 3 Placement (5 rated events), rating history chart, rating shown on profile per track.
- 4 Weekly problem rating recalculation job.

Done when (acceptance criteria)

- Glicko-2 matches reference test vectors.
- Ratings can't be changed by any Practice activity (test).

Paste into Claude Code

```
Do phase V1.1 from docs/SPEC.pdf. Document the rating system in plain language for users.  
Re-read CLAUDE.md first. Finish by running: pnpm lint && pnpm test  
&& pnpm test:e2e, and list anything unfinished or needing my action.
```

Phase V1.2 — Weekly contests

Goal: scheduled contests with leaderboards. (Spec: 1, 7)

Build steps

- 1 Contest model, scheduling in admin, registration, 90-minute window, 4 problems, penalty-time scoring.
- 2 Live leaderboard (WebSockets), country filter, frozen board in the last 15 minutes.
- 3 Integrity system applied to every entry; post-contest editorials and discussion unlocked at the end.
- 4 Rating updates after results are final (after a 24-hour review window).

Done when (acceptance criteria)

- Load test: 2,000 simulated participants with live leaderboard updates.
- Late submissions are rejected (test).

Paste into Claude Code

```
Do phase V1.2 from docs/SPEC.pdf.  
Re-read CLAUDE.md first. Finish by running: pnpm lint && pnpm test  
&& pnpm test:e2e, and list anything unfinished or needing my action.
```

Phase V1.3 — Duels

Goal: fast 1v1 matches. (Spec: 1)

Build steps

- 1 Matchmaking queue by track and rating (± 100 , widening after 30 seconds), best of 3.
- 2 Realtime duel room: both progress bars, run/submit status (not code), countdown, result screen.
- 3 Leaving counts as a loss; repeated leaving triggers short bans; rematch and friend challenges.

Done when (acceptance criteria)

- Two browsers can queue, match and complete a duel in Playwright.
- Disconnect and reconnect within 30 seconds resumes the duel.

Paste into Claude Code

```
Do phase V1.3 from docs/SPEC.pdf. Use the Redis adapter so duels work across API instances.
Re-read CLAUDE.md first. Finish by running: pnpm lint && pnpm test
&& pnpm test:e2e, and list anything unfinished or needing my action.
```

Phase V1.4 — Seasons and leagues

Goal: long-term goals. (Spec: 8)

Build steps

- 1 3-month seasons with league tiers from rating, soft reset at season end, season rewards (cosmetic only).
- 2 Weekly XP leagues of 30 users with promotion and relegation.

Done when (acceptance criteria)

- Season rollover job tested with a controllable clock.

Paste into Claude Code

```
Do phase V1.4 from docs/SPEC.pdf.
Re-read CLAUDE.md first. Finish by running: pnpm lint && pnpm test
&& pnpm test:e2e, and list anything unfinished or needing my action.
```

Phase V1.5 — New languages and tracks

Goal: Java, C++, Bash, plus Code review, Backend and AI-era tracks. (Spec: 4, 5, 6)

Build steps

- 1 Sandbox images for Java, C++ (with compile step and compile limits) and Bash.
- 2 Review format: diff viewer, click lines, choose issue type, graded against planted issues.
- 3 Build format: multi-file projects with a file tree, graded by test suites.
- 4 AI-era challenges: choose the correct of several solutions, find bugs in AI-written code, write tests that catch a planted bug.
- 5 Romanian translation of the interface.

Done when (acceptance criteria)

- Verdict tests for each new language.
- At least 10 validated problems per new track.

Paste into Claude Code

Do phase V1.5 from docs/SPEC.pdf, one language or track at a time, stopping after each for review. Re-read CLAUDE.md first. Finish by running: `pnpm lint && pnpm test && pnpm test:e2e`, and list anything unfinished or needing my action.

Phase V1.6 — Live labs

Goal: Linux, DevOps and live security labs. (Spec: 6.3)

Build steps

- 1 apps/labs orchestrator on separate lab hosts: create, extend, destroy, idle timeout, concurrency caps.
- 2 Terminal gateway (WebSocket + xterm.js) and lab reverse proxy on a separate domain, with short-lived signed tokens.
- 3 Terminal task format with in-lab checkers.
- 4 Linux & DevOps track (10 labs) and live web-security labs (5).

Done when (acceptance criteria)

- A lab can't reach the internet or another user's lab (test).
- Every lab is destroyed after timeout (test).

Paste into Claude Code

Do phase V1.6 from docs/SPEC.pdf. Write the isolation tests first. Re-read CLAUDE.md first. Finish by running: `pnpm lint && pnpm test && pnpm test:e2e`, and list anything unfinished or needing my action.

Phase V1.7 — Profiles and certificates

Goal: turn results into career proof. (Spec: 1)

Build steps

- 1 Public profile pages with privacy controls, open-to-work settings, portfolio links, CV PDF export.
- 2 Certificate exams per track and level (proctored verified sets), public verification pages, 2-year expiry.
- 3 Interview tracks by role and level.

Done when (acceptance criteria)

- Private fields never appear on public pages or APIs (test).

Paste into Claude Code

Do phase V1.7 from docs/SPEC.pdf. Re-read CLAUDE.md first. Finish by running: `pnpm lint && pnpm test && pnpm test:e2e`, and list anything unfinished or needing my action.

Phase V1.8 — Community discussions

Goal: talk about problems safely. (Spec: 3.2, 3.5)

Build steps

- 1 Discussion per problem (Questions, Approaches, Bugs), solution posts, voting, spoiler protection.
- 2 Markdown with strict sanitising, code blocks, mentions.
- 3 Report, block, mute; moderator queue; spam limits for new accounts.

Done when (acceptance criteria)

- XSS test suite on every user field passes.
- Solutions stay hidden until the viewer solved or gave up (test).

Paste into Claude Code

```
Do phase V1.8 from docs/SPEC.pdf.  
Re-read CLAUDE.md first. Finish by running: pnpm lint && pnpm test  
&& pnpm test:e2e, and list anything unfinished or needing my action.
```

Phase V1.9 — Hiring assessments and v1 release

Goal: the first business product, then release v1. (Spec: 10, 3)

Build steps

- 1 Organization, Membership, Assessment and Invite models; org roles; 2FA required for owners.
- 2 Assessment builder from library problems, invite links, candidate flow with consent, results with integrity score and replay (with candidate permission).
- 3 Hiring plan billing in Stripe; student discount.
- 4 Repeat the L12 hardening checklist for all v1 features and deploy.

Done when (acceptance criteria)

- An org can see only its own candidates (test).
- SECURITY_REVIEW.md updated for v1.

Paste into Claude Code

```
Do phase V1.9 from docs/SPEC.pdf.  
Re-read CLAUDE.md first. Finish by running: pnpm lint && pnpm test  
&& pnpm test:e2e, and list anything unfinished or needing my action.
```

PART V2

Version 2

Mobile, creators, social features, the frontend track, more languages, teams and classrooms, AI features, and new competitive formats. Start after the v1 gate: about 5,000 weekly active users and 5 paying companies.

Phase V2.1 — Mobile companion app

Goal: short, daily sessions on the phone. (Spec: 1, 10)

Build steps

- 1 Expo app sharing packages/shared: sign-in, daily challenge, streaks, lessons, predict-output, review and flag formats, review queue as flashcards.
- 2 Quick duels in phone formats (predict-output rounds).
- 3 Push notifications (opt-in) for contests, duels and study groups.
- 4 No in-app purchases; show plan status only.

Done when (acceptance criteria)

- Runs on Android and iOS simulators; e2e tests for sign-in and daily challenge.

Paste into Claude Code

```
Do phase V2.1 from docs/SPEC.pdf. Tell me which store accounts and steps I need.  
Re-read CLAUDE.md first. Finish by running: pnpm lint && pnpm test  
&& pnpm test:e2e, and list anything unfinished or needing my action.
```

Phase V2.2 — Creator program

Goal: community-made content with revenue share. (Spec: 5)

Build steps

- 1 Web problem editor producing valid packages, with live validation.
- 2 Review pipeline: automated checks, reviewer queue, feedback, approval.
- 3 Revenue share accounting from Pro usage, payouts through Stripe Connect.
- 4 Creator courses marketplace with commission.

Done when (acceptance criteria)

- Unapproved problems are never visible to others (test).
- Payout calculations tested with fixtures.

Paste into Claude Code

```
Do phase V2.2 from docs/SPEC.pdf.  
Re-read CLAUDE.md first. Finish by running: pnpm lint && pnpm test  
&& pnpm test:e2e, and list anything unfinished or needing my action.
```

Phase V2.3 — Social features

Goal: study groups, clubs, follows and messages. (Spec: 3.5)

Build steps

- 1 Follow, activity feed (public actions only), study groups (up to 20) with group streaks, clubs with leaderboards.
- 2 Direct messages only between mutual follows or group members; blocking and reporting; under-18 protections.

Done when (acceptance criteria)

- An under-18 user can't be messaged by a stranger (test).

Paste into Claude Code

Do phase V2.3 from docs/SPEC.pdf.
Re-read CLAUDE.md first. Finish by running: `pnpm lint && pnpm test && pnpm test:e2e`, and list anything unfinished or needing my action.

Phase V2.4 — Frontend track

Goal: HTML, CSS and JavaScript challenges. (Spec: 6)

Build steps

- 1 Live preview on a separate sandbox domain, browser tests run with Playwright inside the runner.
- 2 15 frontend problems across difficulties.

Done when (acceptance criteria)

- Preview iframe can't access the main site's cookies or storage (test).

Paste into Claude Code

Do phase V2.4 from docs/SPEC.pdf.
Re-read CLAUDE.md first. Finish by running: `pnpm lint && pnpm test && pnpm test:e2e`, and list anything unfinished or needing my action.

Phase V2.5 — More languages

Goal: Go, C#, Rust, Kotlin and C. (Spec: 6)

Build steps

- 1 Sandbox images and harnesses for each, compile limits, verdict tests.
- 2 Algorithm problems enabled for all; debugging content added per language over time.

Done when (acceptance criteria)

- Verdict test matrix passes for every language.

Paste into Claude Code

Do phase V2.5 from docs/SPEC.pdf, one language at a time.
Re-read CLAUDE.md first. Finish by running: `pnpm lint && pnpm test && pnpm test:e2e`, and list anything unfinished or needing my action.

Phase V2.6 — Teams and classrooms

Goal: training and education products. (Spec: 10)

Build steps

- 1 Team dashboards, assigned learning paths, internal contests.
- 2 Classroom mode: classes, assignments with deadlines, auto-grading, plagiarism similarity check, exam mode using verified attempts, gradebook export.
- 3 Teams and Education billing; SSO with SAML and Google/Microsoft for organisations.

Done when (acceptance criteria)

- A teacher sees only their own classes (test).
- SSO login flow tested with a test identity provider.

Paste into Claude Code

```
Do phase V2.6 from docs/SPEC.pdf.  
Re-read CLAUDE.md first. Finish by running: pnpm lint && pnpm test  
&& pnpm test:e2e, and list anything unfinished or needing my action.
```

Phase V2.7 — AI features (Pro Plus)

Goal: helpful AI that teaches, within a budget. (Spec: 10)

Build steps

- 1 AI gateway service with a provider interface, per-user monthly allowance, daily cap, caching, and cost logging (AiUsage).
- 2 AI tutor in Practice (asks guiding questions, never gives full code unless the user gave up), explain my error, review my solution.
- 3 Hard rule: AI is disabled for any Attempt, contest or duel (enforced in the API).
- 4 Pro Plus plan in Stripe; spending alerts.

Done when (acceptance criteria)

- AI endpoints return PLAN_REQUIRED for Pro and Free (test).
- AI is unreachable during a verified attempt (test).

Paste into Claude Code

```
Do phase V2.7 from docs/SPEC.pdf. Keep provider API keys only on the server, in the secret store.  
Re-read CLAUDE.md first. Finish by running: pnpm lint && pnpm test  
&& pnpm test:e2e, and list anything unfinished or needing my action.
```

Phase V2.8 — New competitive formats

Goal: team battles, CTF events, optimisation arenas, spectating. (Spec: 1)

Build steps

- 1 Team battles (3v3, 5v5) with team matchmaking.
- 2 CTF events (24–48 hours, teams, dynamic scoring).
- 3 Optimisation arenas and king-of-the-hill leaderboards.
- 4 Spectator mode for duels and contest finals, and opt-in public replays.

Done when (acceptance criteria)

- Load test for a 500-team CTF event.

Paste into Claude Code

```
Do phase V2.8 from docs/SPEC.pdf, one format at a time.  
Re-read CLAUDE.md first. Finish by running: pnpm lint && pnpm test  
&& pnpm test:e2e, and list anything unfinished or needing my action.
```

Phase V2.9 — v2 hardening and release

Goal: release v2 safely. (Spec: 3, 11)

Build steps

- 1 Repeat the full L12 checklist for all v2 features.
- 2 Mobile app store submissions.
- 3 Update privacy policy and data safety declarations for new data types.

Done when (acceptance criteria)

- SECURITY_REVIEW.md updated with no open high/critical items.

Paste into Claude Code

```
Do phase V2.9 from docs/SPEC.pdf.  
Re-read CLAUDE.md first. Finish by running: pnpm lint && pnpm test  
&& pnpm test:e2e, and list anything unfinished or needing my action.
```

12. What you must do yourself

Task	When
Name, domain, company registration, bank account	L0
GitHub, Stripe, email provider, hosting accounts	L0
Review every drafted problem and lesson before publishing	L6, L11, and ongoing
Lawyer review of privacy policy, terms and company contracts	L12, V1.9
Production servers, DNS, email domain records, Stripe live mode	L13
Recruit beta testers; talk to users and companies	Before and after launch
Moderation and support until you can hire help	From launch
App store developer accounts	V2.1
Pay infrastructure costs: VMs, storage, email, Stripe fees; AI costs in v2	Ongoing

13. Decisions made for you

Change any of these before the phase that uses them and tell Claude Code.

Decision	Default	Why
Name	Forge (placeholder)	Likely taken in some form; choose before L0
Stack	TypeScript monorepo, Next.js, NestJS, PostgreSQL	One language, strong ecosystem, easy hiring
Sandbox	Docker + gVisor on separate VMs	Strong isolation without running your own VM fleet
Payments	Stripe	Subscriptions, EU VAT and invoices built in
Launch languages	Python, JavaScript/TypeScript, SQL	Covers most learners and interviews
Launch tracks	Algorithms, Debugging, SQL, Security (static)	Mix of classic demand and the differentiator
Rating system	Glicko-2	Handles new and inactive players better than Elo
Verified limits on Free	3 per week	Keeps costs and abuse down while letting everyone get verified
Minimum age	16	Romania's age of digital consent
Prices	Pro €12/month, €99/year	Adjust after interviews with users

Appendix A — CLAUDE.md

Save this as CLAUDE.md in the repository root. Claude Code reads it at the start of every session.

CLAUDE.md

Forge - rules for Claude Code

The full specification is docs/SPEC.pdf. Read the sections a phase references before starting. If this file and the spec disagree, stop and ask.

How to work

- One phase at a time, in order. Never start a phase I didn't ask for.
- Before coding: restate the phase goal, list files to create/change, list risks.
- Tests first for security-sensitive work (auth, permissions, runner, billing, integrity).
- A phase is done only when lint, type-check, unit, integration and e2e tests all pass. Show the command output.
- End every phase with: what was built, what's untested, what needs my manual action.
- Ask before adding a dependency not named in the spec. Use current stable versions only; look them up, never invent version numbers, no pre-releases.
- Small commits with clear messages. Record significant decisions in docs/decisions/.

Security rules (never break)

- No secrets in git. .env files are git-ignored; document variables in .env.example.
- Authorization is checked in the API for every resource. The UI never decides access.
- Plan limits only through EntitlementsService and packages/shared/plans.ts.
- Validate every input with zod schemas from packages/shared.
- Untrusted code runs only in apps/runner sandboxes on isolated hosts. Never exec user code in the API or web app.
- Hidden tests, reference solutions and competitive templates never leave the server.
- Sanitize all user-generated markdown. No raw HTML rendering.
- AI features are never available during verified attempts, contests or duels.
- Never log passwords, tokens, session IDs, full emails or code from verified attempts.

Conventions

- TypeScript strict everywhere. No `any` without a comment explaining why.
- API errors: { error: { code, message, details? } } with stable codes.
- Money in integer minor units with currency codes. Times in UTC.
- UI: shadcn/ui components and tokens from packages/ui; dark and light themes; WCAG 2.2 AA.
- Content: problem packages must pass `pnpm problems:validate`; drafted content gets status: needs-review until I approve it.

Commands

- pnpm dev start everything locally
- pnpm lint && pnpm test lint, types, unit and integration tests
- pnpm test:e2e Playwright tests
- pnpm problems:validate validate all content packages